The Islamia University of Bahawalpur

Department of Computer Science



Final Year Project Documentation

for

A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases

By

Bilal Ilyas Jhandir

BS Computer Science (M)

S22BDOCS1M01001

Session Spring 2022 – 2026

Supervisor

Mr. Syed Naseem Abbas Naqvi

A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases

Bilal Ilyas Jhandir
BS Computer Science
S22BDOCS1M01001
Session Spring 2022 - 2026

[Project Documentation]

Version: 1.0 **Date: (1/9/2026)**

Table of Contents

Chapter 1 Introduction **5**

1.1 Purpose *5*

1.2 Scope *5*

1.3 Definitions, Acronyms, and Abbreviations. *6*

1.4 References *6*

1.5 Overview *7*

1.6 Product Perspective *9*
1.6.1 System Interfaces 10
1.6.2 Interfaces 10
1.6.3 Hardware Interfaces 10
1.6.4 Software Interfaces 11
1.6.5 Communications Interfaces 11
1.6.6 Memory Constraints 11
1.6.7 Operations 11
1.6.8 Site Adaptation Requirements 12

1.7 Product Functions *12*

1.8 User Characteristics *12*

1.9 Constraints *12*

1.10 Assumptions and Dependencies *13*

1.11 Apportioning of Requirements *13*

Chapter 2 Software Requirements Specification **15**

2.1 Specific Requirements *15*

2.2 External Interfaces *15*

2.3 Functions *16*

2.4 Performance Requirements *16*

2.5 Logical Database Requirements *17*

2.6 Design Constraints *17*
2.6.1 Standards Compliance *18*

2.7 Software System Attributes *18*
2.7.1 Reliability *19*
2.7.2 Availability *19*
2.7.3 Security *19*

2.7.4 Maintainability 20
2.7.5 Portability 20
2.8 Organizing the Specific Requirements 19
2.8.1 System Mode 21
2.8.2 User Class 21
2.8.3 Objects 21
2.8.4 Feature 21
2.8.5 Stimulus 21
2.8.6 Response 21
2.8.7 Functional Hierarchy 21
2.9 Additional Comments 19

Chapter 3 System Design and Architecture **21**

3.1 Architectural Design 21
3.2 Decomposition Description 21
3.3 Design Rationale 23
3.4 Data Description 24
3.5 Data Dictionary 25
3.6 COMPONENT DESIGN 25

Chapter 4 Implementation and Testing **29**

4.1 Code Modules 29
4.2 Database Connectivity 30
4.3 Overview of User Interface 31
4.4 Screen Images 31
4.5 Screen Objects and Actions 37
4.6 Test Cases 39

Chapter 1
Introduction

The following subsections of the Software Requirements Specifications (SRS) document provide an overview of the entire SRS. This document outlines the functional and non-functional requirements for the "A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases" so that system designers and developers can implement the solution effectively.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for "A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases". This document covers the system's intended features, interface requirements, and constraints.

The intended audience for this document includes:

- **Project Supervisors and Evaluators:** To verify that the proposed system meets the academic and technical standards of the final year project.
- **Software Developers/Data Scientists:** To understand the functional requirements for model integration and application development.
- **Medical Professionals:** To understand the system's capabilities and how it serves as a diagnostic aid.

1.2 Scope

(1) Product Name: A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases**.**

(2) Product Scope: The software is an automated diagnostic tool designed to analyze chest X-ray images. It classifies images into one of seven distinct categories: **Atelectasis, Bacterial Pneumonia, COVID-19, Emphysema, Tuberculosis, Viral Pneumonia, and Normal.**

The system **will**:

- Accept chest X-ray images as input from local storage.

- Pre-process images (resizing, normalization) to match model requirements.
- Utilize trained Deep Learning models to predict the disease class.
- Provide a confidence score (probability) for the prediction.
- Generate visual explanations using Grad-CAM heatmaps to localize the affected regions in the lung.
- Display a superimposed image (Original X-ray + Heatmap) for clinical review.

The system **will not**:

- Make final medical decisions or prescribe medication. It is strictly a decision-support tool.
- Directly control X-ray hardware or radiation equipment.

(3) Application and Benefits: The application is specified for use in medical diagnostic centers and hospitals to assist radiologists. The primary benefit is the reduction of diagnosis time and the mitigation of human error, particularly in cases where disease patterns are subtle. It addresses the critical need for rapid screening during outbreaks (e.g., COVID-19) and helps manage high patient loads.

(4) Consistency: This scope is consistent with the project proposal approved for the Final Year Project, focusing on leveraging Computer Vision for medical imaging.

1.3 Definitions, Acronyms, and Abbreviations.

- **SDD:** Software Design Document.
- **SRS:** Software Requirements Specification.
- **DL:** Deep Learning.
- **CNN:** Convolutional Neural Network.
- **GUI:** Graphical User Interface.
- **Grad-CAM:** Gradient-weighted Class Activation Mapping (a technique for visualizing model focus).
- **Dataset:** A collection of 42,000+ X-ray images used for training and testing.
- **Inference:** The process of using a trained model to make predictions on new data.
- **Heatmap:** A graphical representation of data where values are depicted by color, used here to show disease localization.

1.4 References

(1) Documents Referenced:

1. **Ayalew, A. M., Bezabh, Y. A., Abuhayi, B. M., Ayalew, A. Y., (2024).** *Atelectasis detection in chest X-ray images using convolutional neural networks and transfer learning with anisotropic diffusion filter*. *Informatics in Medicine Unlocked, 45*, Article 101448. https://doi.org/10.1016/j.imu.2024.101448
2. **Huang, X., Li, B., Huang, T., Yuan, S., Wu, W., Yin, H., & Lyu, J. (2022).** External validation based on transfer learning for diagnosing atelectasis using portable chest X-rays. *Frontiers in Medicine, 9*, 920040. https://doi.org/10.3389/fmed.2022.920040
3. **Alazab, M., Awajan, A., Mesleh, A., Abraham, A., Jatana, V., & Alhyari, S. (2020).** COVID-19 prediction and detection using deep learning. *International Journal of Computer Information Systems and Industrial Management Applications, 12*, 168–181.
4. **Ayalew, A. M., Salau, A. O., Tamyalew, Y., Abeje, B. T., & Woreta, N. (2023).** X-ray image-based COVID-19 detection using deep learning. *Multimedia Tools and Applications, 82*, 44507–44525. https://doi.org/10.1007/s11042-023-15389-8
5. **Ayalew, A. M., Salau, A. O., Abeje, B. T., & Enyew, B. (2022).** Detection and classification of COVID-19 disease from X-ray images using convolutional neural networks and histogram of oriented gradients. *Biomedical Signal Processing and Control, 74*, 103530. https://doi.org/10.1016/j.bspc.2022.103530
6. **Sharma, V., Nillmani, Gupta, S. K., & Shukla, K. K. (2023).** Deep learning models for tuberculosis detection and infected region visualization in chest X-ray images. *Intelligent Medicine, 4*(2), 104–113. https://doi.org/10.1016/j.imed.2023.06.001
7. **Hansun, S., Argha, A., Liaw, S. T., Celler, B. G., & Marks, G. B. (2023).** Machine and deep learning for tuberculosis detection on chest X-rays: Systematic literature review. *Journal of Medical Internet Research, 25*, e43154. https://doi.org/10.2196/43154
8. **Alsaffar, M., Alshammari, G., Alshammari, A., Aljaloud, S., Almurayziq, T. S., Hamad, A. A., Kumar, V., & Belay, A. (2021).** Detection of tuberculosis

disease using image processing technique. *Mobile Information Systems, 2021*, Article 7424836. https://doi.org/10.1155/2021/7424836

9. **Sharma, S., & Guleria, K. (2023).** A deep learning based model for the detection of pneumonia from chest X-ray images using VGG-16 and neural networks. *Procedia Computer Science, 218*, 357–366. https://doi.org/10.1016/j.procs.2023.01.018
10. **Varshni, D., Thakral, K., Agarwal, L., Nijhawan, R., & Mittal, A. (2019).** Pneumonia detection using CNN-based feature extraction. In *Proceedings of the 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)* (pp. 1–7). IEEE. https://doi.org/10.1109/ICECCT.2019.8869364
11. **Gu, X., Pan, L., Liang, H., & Yang, R. (2018).** Classification of bacterial and viral childhood pneumonia using deep learning in chest radiography. In *Proceedings of the 3rd International Conference on Multimedia and Image Processing (ICMIP '18)* (pp. 88–93). Association for Computing Machinery. https://doi.org/10.1145/3195588.3195597
12. **Abiyev, R. H., & Ma'aitah, M. K. S. (2018).** Deep convolutional neural networks for chest diseases detection. *Journal of Healthcare Engineering, 2018*, Article 4168538. https://doi.org/10.1155/2018/4168538
13. **Ash, S. Y., Choi, B., Oh, A., Lynch, D. A., & Humphries, S. M. (2023).** Deep learning assessment of progression of emphysema and fibrotic interstitial lung abnormality. *American Journal of Respiratory and Critical Care Medicine, 208*(6), 666–675. https://doi.org/10.1164/rccm.202211-2098OC
14. **Humphries, S. M., Notary, A. M., Centeno, J. P., Strand, M. J., Crapo, J. D., Silverman, E. K., Lynch, D. A., & COPDGene Investigators. (2019).** Deep learning enables automatic classification of emphysema pattern at CT. *Radiology, 294*(2), 434–444. https://doi.org/10.1148/radiol.2019191022

(2) Sources:

1. https://www.kaggle.com/datasets/tolgadincer/labeled-chest-xray-images
2. https://www.kaggle.com/datasets/nabeelsajid917/covid-19-x-ray-10000-images
3. https://www.kaggle.com/datasets/nikhilpandey360/chest-xray-masks-and-labels
4. https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia
5. https://www.kaggle.com/nih-chestxrays/data

6. https://www.kaggle.com/datasets/fsang2692/lung-area-specific-covid19-xray-dataset
7. https://www.kaggle.com/datasets/ankitachoudhury01/covid-patients-chest-xray
8. https://www.kaggle.com/datasets/thomasnibb/image-classification-covid19-xray
9. https://www.kaggle.com/datasets/mohamedasak/chest-x-ray-6-classes-dataset
10. https://data.mendeley.com/datasets/8j2g3csprk/2
11. Dataset of 1011 images from local hospital named BVH
12. **Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022).** *A ConvNet for the 2020s* (arXiv:2201.03545). arXiv. https://doi.org/10.48550/arXiv.2201.03545
13. **Ilham, W., & Ahmad, A. (2025).** A comprehensive review of ConvNeXt architecture in image classification: Performance, applications, and prospects. *International Journal of Advanced Computing and Informatics, 2*(2), 108–114. https://doi.org/10.71129/ijaci.v2i2.pp108-114
14. **Tan, M., & Le, Q. V. (2021).** *EfficientNetV2: Smaller models and faster training* (arXiv:2104.00298). arXiv. https://doi.org/10.48550/arXiv.2104.00298
15. **Zhao, H., Zhang, X., Gao, Y., Wang, L., Xiao, L., Liu, S., Huang, B., & Li, Z. (2024).** Diagnostic performance of EfficientNetV2-S method for staging liver fibrosis based on multiparametric MRI. *Heliyon, 10*, e35115. https://doi.org/10.1016/j.heliyon.2024.e35115

1.5 Overview

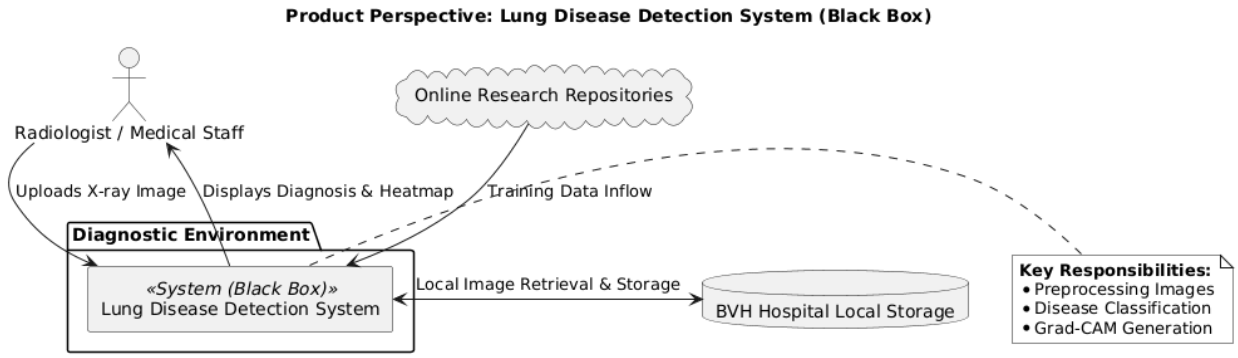
(1) Content Description: The remainder of this SRS describes the specific requirements for the system. **Chapter 2** details the specific functional and performance requirements, including user interfaces and system attributes. **Chapter 3** covers the system design and architecture, including data flow and component decomposition. **Chapter 4** discusses implementation details, testing strategies, and validation results.

(2) Organization: This document is organized by creating a high-level context in Chapter 1, followed by detailed specific requirements in Chapter 2, and finally technical design and implementation in subsequent chapters.

1.6 Product Perspective

The "A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases" is a research-oriented software product designed to function as a standalone diagnostic aid. While currently a self-contained prototype, it is designed with the perspective of being a module that could eventually integrate into a hospital's Picture Archiving and Communication System (PACS).

Unlike general image classifiers, this system is specialized for medical imaging, incorporating specific techniques to handle Class Imbalance (e.g., rare diseases like Atelectasis vs. common Normal cases) and Visual Localization (Grad-CAM).



The following subsections describe how the software operates inside various constraints.

1.6.1 System Interfaces

The system acts as a processing engine that interacts with:

- **File System:** To retrieve X-ray images and store generated reports/heatmaps.
- **User Interface:** A software interface (Web or Desktop based) that allows the radiologist to upload images and view results.
- **Input:** Single or Batch X-ray images (JPEG/PNG/DICOM format).
- **Output:** Classification labels, probability scores, and visualized heatmaps.

1.6.2 Interfaces

(1) Logical Characteristics: The user interface is designed for simplicity. The user can:

- Select an image source.
- View the prediction (e.g., "COVID-19 Detected") alongside a confidence percentage (e.g., "98%").
- Toggle the Grad-CAM visualization overlay to see *why* the model made the prediction.

(2) Optimization: The interface is optimized for medical professionals who may not have deep technical knowledge of AI. The terminology used in the display is medical (e.g., "Bacterial Pneumonia") rather than technical (e.g., "Class 1").

1.6.3 Hardware Interfaces

The software does not directly control physical hardware (like X-ray machines). However, it requires specific hardware to run the inference engine efficiently:

- **GPU Interface:** The system interfaces with NVIDIA GPUs (specifically optimized for NVIDIA Tesla T4 or similar) via CUDA drivers to accelerate image processing and model inference.
- **Display:** Standard high-resolution monitors for viewing X-rays and heatmaps.

1.6.4 Software Interfaces

The system relies on the following external software components:

- **Operating System:** Linux (Ubuntu) or Windows.
- **Runtime Environment:** Python 3.8+.
- **Deep Learning Framework:** PyTorch (Version 1.10+). The system interacts with PyTorch APIs to load the .pth model checkpoints.
- **Image Processing:** OpenCV and Pillow libraries for image transformation (resizing to 224x224, normalization).
- **CUDA Toolkit:** For GPU acceleration.

1.6.5 Communications Interfaces

- **Local/Network Access:** If deployed as a web application, the system uses standard HTTP/HTTPS protocols to transmit image data from the client to the inference server.
- **Internal Data Bus:** Standard PCIe communication between the CPU and GPU for transferring image tensors.

1.6.6 Memory Constraints

- **VRAM (Video RAM):** The Deep Learning models (EfficientNetV2-L and ConvNeXt-Large) are memory-intensive. The target hardware must have a minimum of 8GB VRAM (recommended 16GB, such as the Tesla T4 used in development) to handle batch processing without "Out of Memory" errors.
- **RAM:** A minimum of 16GB system RAM is required to load the dataset generators and handle image preprocessing.

1.6.7 Operations

- **Interactive Mode:** The standard mode where a user manually uploads an image and waits for the result (Real-time inference).
- **Batch Mode:** A background operation where the system processes a folder of images (e.g., the BVH test set) and generates a classification report or confusion matrix.
- **Maintenance:** Periodic updates to the .pth model files if the model is retrained with new data.

1.6.8 Site Adaptation Requirements

- **Environment Setup:** The deployment site must have the correct Python environment and NVIDIA drivers installed.
- **Data Privacy:** If installed at a hospital site, the system must be configured to access data in compliance with patient privacy regulations, ensuring no patient names are stored in the training logs.

1.7 Product Functions

The major functions of the software are:

1. **Image Ingestion:** Loading X-ray images from disk or user upload.
2. **Preprocessing:** Automatically resizing images to 224x224 pixels, converting to RGB, and normalizing pixel values to standard distributions.
3. **Disease Classification:** Analyzing the processed image using the "Best Model" (highest validation accuracy checkpoint) to categorize it into one of the 7 classes.
4. **Localization (Explainability):** Generating a Grad-CAM heatmap to highlight the regions of interest (ROI) that contributed to the prediction.
5. **Result Visualization:** Creating a superimposed image merging the original X-ray with the heatmap.
6. **Performance Reporting:** (In Batch Mode) Calculating and displaying Precision, Recall, F1-Score, and Confusion Matrices for the processed dataset.

1.8 User Characteristics

- **Primary User (Radiologist/Doctor):** High domain expertise in medicine/lung anatomy. Low technical expertise in AI/Python. They require a clear, visual output and reliable accuracy.

- **Secondary User (Administrator/Researcher):** High technical expertise. Capable of interpreting loss curves, updating model weights, and managing the Python environment.

1.9 Constraints

- **Hardware Dependencies:** The system requires NVIDIA GPUs for reasonable performance. Running on CPU alone will result in significant latency (slow inference time).
- **Data Imbalance:** The real-world data (and the training data) is naturally imbalanced (e.g., fewer Tuberculosis cases than Normal cases). The system logic must account for this to avoid bias (addressed via Weighted Random Sampling in the model logic).
- **Image Quality:** The system requires X-rays of reasonable quality. Extreme rotation, blurring, or artifacts in the input image may degrade prediction accuracy.
- **Black Box Nature:** While Grad-CAM provides some explainability, deep learning models are inherently "black boxes," which acts as a constraint on complete clinical trust without human verification.

1.10 Assumptions and Dependencies

- **Assumption:** It is assumed that the input images are Chest X-rays (PA/AP view). The system is not designed to detect anomalies in other body parts (e.g., bone fractures in hands).
- **Assumption:** The X-ray images provided are in standard formats (JPG, PNG).
- **Dependency:** The system depends on the availability of the PyTorch library and compatible CUDA drivers on the host machine.
- **Dependency:** The accuracy of the system is dependent on the quality and diversity of the dataset collected from online sources and BVH.

1.11 Apportioning of Requirements

- **Current Version:** Includes training on the 7-class dataset, static image classification, Grad-CAM visualization, and performance metrics generation.

- **Future Versions:** May include integration with live hospital PACS servers (DICOM communication), a mobile application for remote diagnosis, and real-time segmentation of lung nodules (pixel-level masking rather than just heatmaps).

Chapter 2
Software Requirements Specification

2.1 Specific Requirements

This section outlines the detailed functional and non-functional requirements for the A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases. These requirements are derived from the objective to accurately classify chest X-rays into seven distinct categories (Atelectasis, Bacterial Pneumonia, COVID-19, Emphysema, Tuberculosis, Viral Pneumonia, Normal) using EfficientNetV2-L and ConvNeXt-Large architectures.

2.2 External Interfaces

2.2.1 User Interface

- **Name:** Main Diagnostic Dashboard.
- **Description:** A Graphical User Interface (GUI) allowing users to interact with the underlying deep learning models.
- **Screen Organization:**
    - **Input Panel:** A specific zone (drag-and-drop or file browser) for selecting X-ray images.
    - **Result Panel:** A display area showing the "Predicted Class" and "Confidence Score."
    - **Visualizer Panel:** A toggleable view to switch between the Original X-ray, the Heatmap, and the Superimposed Image.
- **End Messages:** Clear text indicators for "Processing...", "Diagnosis Complete," or "Error: Invalid File Format."

2.2.2 System Input: X-ray Image Data

- **Name:** Input X-ray Image.
- **Source:** Local file system (Hard Drive / USB) or direct camera capture (if integrated).
- **Valid Range:**
    - **Format:** JPG, PNG, or DICOM.

    - **Dimensions:** Minimum 224x224 pixels.
    - **Channels:** Grayscale (1-channel) or RGB (3-channel).
- **Units of Measure:** Pixel intensity (0-255).
- **Timing:** Real-time upload.

2.2.3 System Output: Diagnostic Report

- **Name:** Classification Result.
- **Destination:** User Interface Result Panel.
- **Content:**
    - **Class Label:** One of the 7 disease categories.
    - **Probability:** A percentage value (0-100%) indicating model certainty.
    - **Grad-CAM Heatmap:** A color-coded overlay where 'Red' indicates high model attention and 'Blue' indicates low attention.

2.3 Functions

This section defines the fundamental actions the system must perform.

2.3.1 Image Preprocessing Function

- **REQ-2.3.1.1:** The system shall accept X-ray images of varying resolutions as input.
- **REQ-2.3.1.2:** The system shall automatically resize all input images to a fixed resolution of **224x224 pixels** to match the input layer of the EfficientNetV2-L and ConvNeXt-Large models.
- **REQ-2.3.1.3:** The system shall normalize pixel values using the mean and standard deviation specific to the training dataset (ImageNet standards) to ensure consistent model inference.

2.3.2 Inference Function

- **REQ-2.3.2.1:** The system shall load the pre-trained weights from the .pth checkpoint file designated as "Best Model."
- **REQ-2.3.2.2:** The system shall perform a forward pass of the preprocessed image tensor through the neural network.
- **REQ-2.3.2.3:** The system shall calculate the Softmax probability distribution across the 7 output nodes.
- **REQ-2.3.2.4:** The system shall identify the class with the highest probability score as the final prediction.

2.3.3 Visualization Function (Explainability)

- **REQ-2.3.3.1:** The system shall generate a Grad-CAM (Gradient-weighted Class Activation Mapping) heatmap for the predicted class.
- **REQ-2.3.3.2:** The system shall generate a "Superimposed Result" by overlaying the heatmap onto the original input image with 50% opacity to allow clinical localization of the disease.

2.3.4 Batch Evaluation Function

- **REQ-2.3.4.1:** The system shall provide a mode to process a directory of images (e.g., the BVH test set) sequentially without user intervention.
- **REQ-2.3.4.2:** Upon completion of a batch, the system shall generate a Classification Report containing Precision, Recall, and F1-Score for all classes.
- **REQ-2.3.4.3:** The system shall generate a Confusion Matrix image visualizing the count of True Positives vs. False Positives.

2.4 Performance Requirements

2.4.1 Static Requirements

- **REQ-2.4.1.1:** The system shall support a dataset registry of at least 50,000 images without database corruption.
- **REQ-2.4.1.2:** The system shall handle the simultaneous loading of two large-scale Deep Learning models (approx. 500MB+ parameter size each) into memory.

2.4.2 Dynamic Requirements

- **REQ-2.4.2.1: Inference Latency:** On a system equipped with an NVIDIA Tesla T4 GPU (or equivalent), the system shall render a prediction for a single image in less than **2.0 seconds**.
- **REQ-2.4.2.2: Accuracy:** The system shall maintain a validation accuracy of no less than **90%** (based on the reported 90.72% for EfficientNetV2-L) for the defined test dataset.
- **REQ-2.4.2.3: Throughput:** In batch processing mode, the system shall process at least 30 images per second (Batch Size = 32).

2.5 Logical Database Requirements

The system utilizes a file-system-based data management approach rather than a complex relational database.

- **Entities:**
    - **Images:** Stored in directory structures labeled by disease class (e.g., /dataset/covid-19/).
    - **Logs:** CSV files storing training history (Epoch, Loss, Accuracy).
- **Data Retention:** The system shall retain generated Confusion Matrices and Performance Reports until manually deleted by the user.
- **Integrity:** The system shall verify the integrity of image files (check for non-zero file size and valid headers) before attempting processing to prevent runtime crashes.

2.6 Design Constraints

2.6.1 Standards Compliance

- **REQ-2.6.1.1:** The system shall adhere to **HIPAA** (or local equivalent) guidelines regarding patient anonymity; no patient names or PII (Personally Identifiable Information) shall be embedded in the filename or analysis logs of the BVH dataset.
- **REQ-2.6.1.2:** The system output reports shall be formatted to allow easy conversion to PDF for medical record attachment.

2.6.2 Software/Hardware Constraints

- **REQ-2.6.2.1:** The system requires a Python 3.8+ runtime environment.
- **REQ-2.6.2.2:** The system relies on the PyTorch framework; therefore, the host machine must support compatible CUDA drivers for GPU acceleration.

2.7 Software System Attributes

2.7.1 Reliability

- **REQ-2.7.1.1:** The system shall handle "Out of Distribution" images (e.g., an image of a hand instead of a lung) by either flagging them as "Unknown" or providing a low confidence score, rather than crashing.
- **REQ-2.7.1.2:** The system shall not crash if a corrupt image file is encountered during batch processing; it shall skip the file and log an error.

2.7.2 Availability

- **REQ-2.7.2.1:** The system is designed for on-demand use. It shall be available for operation 99.9% of the time provided the host hardware is powered on.

- **REQ-2.7.2.2:** In the event of a system crash, the system shall be capable of restarting and returning to a ready state in under 60 seconds.

2.7.3 Security

- **REQ-2.7.3.1:** Access to the "Training" module (which can alter model weights) shall be restricted to users with administrator privileges (if deployed in a multi-user environment).
- **REQ-2.7.3.2:** Input images from the BVH hospital dataset must be sanitized (metadata removal) before being stored in the project directory.

2.7.4 Maintainability

- **REQ-2.7.4.1:** The code shall be modular; the Model Architecture definition, Data Loading logic, and Training Loop must exist in separate script files or classes.
- **REQ-2.7.4.2:** Hardcoded paths for datasets shall be avoided; the system shall use configuration files (e.g., config.yaml) for setting paths, learning rates, and batch sizes.

2.7.5 Portability

- **REQ-2.7.5.1:** The system shall be deployable on any Operating System (Windows 10/11, Ubuntu 20.04+) that supports Python and PyTorch.
- **REQ-2.7.5.2:** All library dependencies shall be listed in a requirements.txt file to facilitate automated environment setup.

2.7.6 Quality Characteristic Trade-off Matrix The following table illustrates the relative priority of non-functional requirements for this specific research project. "Accuracy" (part of Reliability) is prioritized over "Efficiency" (Speed).

2.8 Organizing the Specific Requirements

The requirements for this system are organized by **Functional Hierarchy**. This organization best reflects the pipeline nature of a Deep Learning application, where data flows through specific processing stages.

- **2.8.1 Data Ingestion:** Requirements related to loading, augmenting, and splitting the dataset (including Handling Class Imbalance).
- **2.8.2 Model Processing:** Requirements related to the specific architectures (EfficientNetV2-L / ConvNeXt-Large) and hyperparameter configuration.
- **2.8.3 Analysis & Visualization:** Requirements related to Grad-CAM generation and heatmap overlay.

- **2.8.4 Reporting:** Requirements related to generating metrics (Confusion Matrix, F1-Score) and validating against the BVH dataset.

2.9 Additional Comments

While standard software systems are deterministic (Input A always equals Output B), this system utilizes Deep Learning, which is probabilistic. Therefore, "Correctness" in this SRS is defined by statistical performance metrics (Accuracy, F1-Score) on a validation set, rather than absolute correctness for every single input case. The inclusion of Grad-CAM (Visual Localization) is a specific requirement intended to mitigate the "Black Box" nature of neural networks by providing interpretable visual feedback to the radiologist.

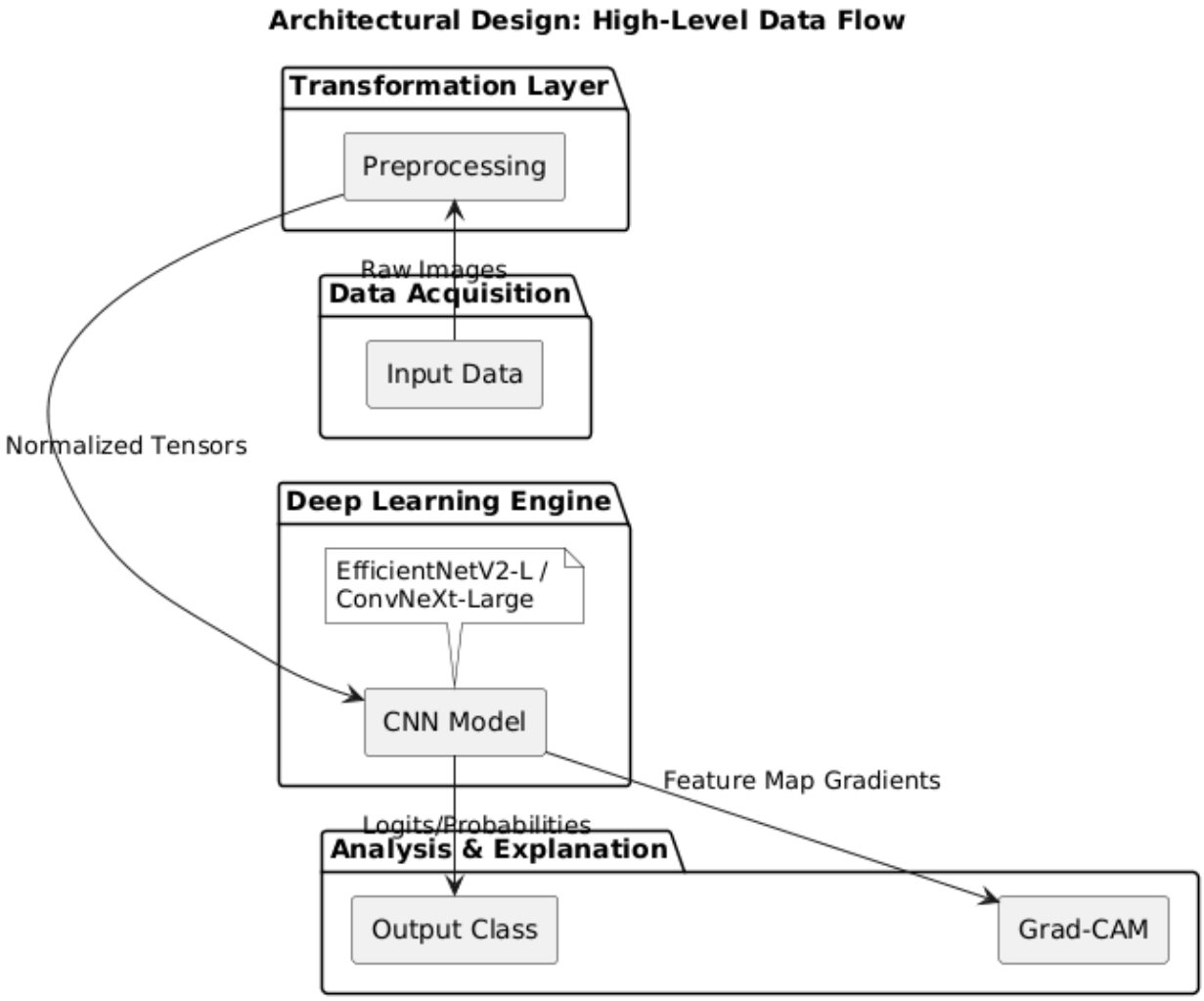
Chapter 3
System Design and Architecture

3.1 Architectural Design

The architecture of the "A Novel Deep Learning-Based System for Identification and Differentiation of Lung Diseases" follows a **Modular Pipeline Architecture**. This design was chosen because deep learning projects rely on distinct, sequential stages of data transformation—from raw image ingestion to probability output. The system is partitioned into four major subsystems: the **Data Processing Subsystem**, the **Model Engine**, the **Training & Validation Subsystem**, and the **Visualization/Inference Interface**.

Collaborative Flow:

1. **Data Processing Subsystem:** Responsible for ingesting raw images from the disk (Online + BVH datasets), applying augmentation, and converting them into normalized tensors. It acts as the "feeder" for the model.
2. **Model Engine:** This is the core computational unit containing the architecture definitions for **EfficientNetV2-L** and **ConvNeXt-Large**. It accepts tensors and outputs raw logits.
3. **Training & Validation Subsystem:** This module manages the learning process. It calculates error (Loss), updates model weights (Backpropagation), and validates performance using the 10% split. It saves the "Best Model" based on validation accuracy.
4. **Visualization Interface:** This subsystem handles post-processing. It takes the trained model and new images to generate diagnostic predictions and Grad-CAM heatmaps for the user.



3.2 Decomposition Description

The system is decomposed functionally into the following modules.

3.2.1 Data Module

- **Dataset Loader:** Iterates through the directory structure where images are stored by class folder.
- **Augmentor:** Applies transformations (Resize, Horizontal Flip, Rotation, Color Jitter) to increase dataset diversity.
- **Sampler:** Implements **Weighted Random Sampling** to handle the class imbalance between common classes (e.g., Normal) and rare classes (e.g., Atelectasis).

3.2.2 Model Architecture Module

- **Feature Extractor:** The convolutional base (backbone) of EfficientNetV2-L or ConvNeXt-Large that extracts high-level patterns from images.

- **Classifier Head:** The final fully connected layers that map extracted features to the 7 specific disease classes.

3.2.3 Evaluation Module

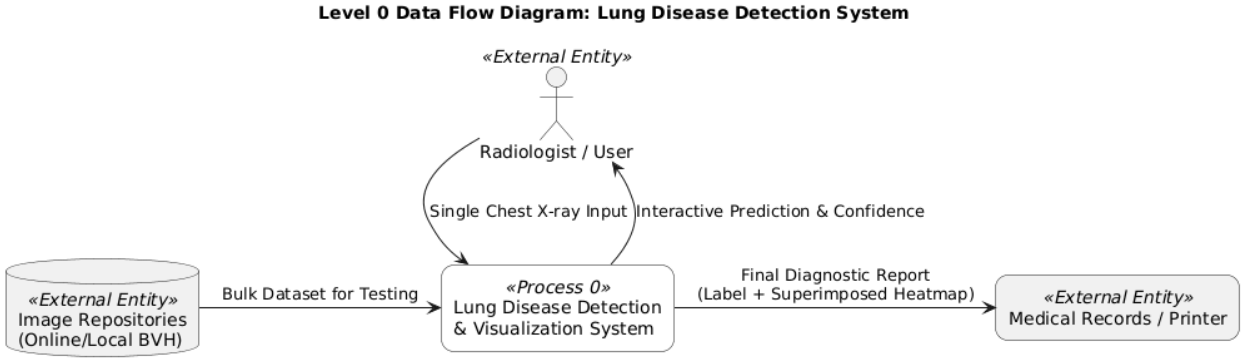
- **Metric Calculator:** Computes Precision, Recall, and F1-Score.
- **Confusion Matrix Generator:** Compares predicted labels against ground truth labels to visualize errors.

3.2.4 Visualization Module

- **Grad-CAM Generator:** Computes the gradients of the target class with respect to the final convolutional layer to produce a localization map.
- **Overlay Utility:** Merges the heatmap with the original image.

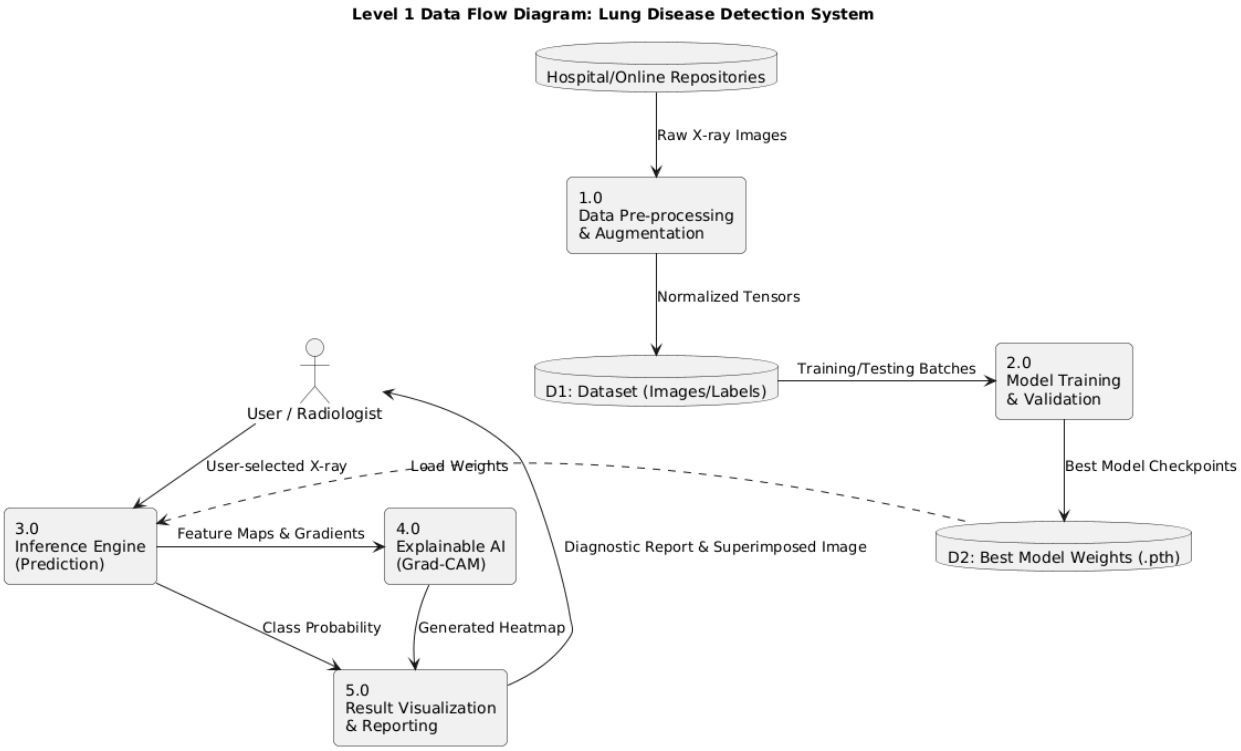
Level 0 DFD (Context Diagram)

This diagram shows the system as a single process interacting with the external entity (the User).



Level 1 DFD (Functional Decomposition)

This diagram breaks down the internal "System" process into its functional components to show how a raw image is transformed into a final prediction.



3.3 Design Rationale

The architecture was selected to prioritize **Accuracy** and **Explainability** over lightweight deployment speed, given the critical medical nature of the application.

- **Choice of Models (EfficientNetV2-L & ConvNeXt-Large):**
  - *Rationale:* We chose **EfficientNetV2-L** because it offers faster training speed and better parameter efficiency compared to older models like ResNet, while maintaining state-of-the-art accuracy. **ConvNeXt-Large** was chosen as a comparative model because it modernizes standard CNNs to compete with Vision Transformers, offering robustness on large datasets.
  - *Trade-off:* These are "Large" variants. They require significant VRAM (Tesla T4 GPUs were required) and have higher inference latency than "Mobile" versions, but the trade-off is justified by the higher sensitivity required for detecting diseases like Tuberculosis.
- **Handling Class Imbalance (Weighted Random Sampling):**

    - *Rationale:* Simply deleting data (Undersampling) would lose valuable information from the "Normal" class. Simply duplicating data (Oversampling) causes overfitting. Weighted Random Sampling was selected as it dynamically balances the batch during training, ensuring the model sees rare diseases frequently enough to learn them.
- **Grad-CAM Integration:**
    - *Rationale:* A standard "Black Box" classifier is insufficient for clinical trust. The architecture explicitly includes a gradient access hook to generate heatmaps, allowing radiologists to verify *where* the model is looking.

3.4 Data Description

The information domain (medical images) is transformed into numerical structures optimized for Matrix Multiplication on GPUs.

- **Input Entity:** * **Raw:** Single-channel (Grayscale) or 3-Channel (RGB) image files (JPG/PNG).
    - **Processed:** Transformed into a Float32 Tensor of shape (Batch_Size, Channels, Height, Width). For this system: (32, 3, 224, 224).
- **Label Entity:** * Stored as an Integer (0 to 6) representing the index of the class (e.g., 0=Atelectasis, 2=COVID-19).
- **Storage:** * Data is stored in a hierarchical directory structure. No complex SQL database is required for the images themselves; the file system serves as the database.
    - Model Weights are stored as .pth binary files.

3.5 Data Dictionary

Alphabetical list of system entities and major objects:

- **Batch:** A collection of 32 images processed simultaneously by the GPU.
- **ClassMap:** A dictionary mapping integer indices to human-readable strings (e.g., {0: 'Atelectasis', 1: 'Bacterial Pneumonia', ...}).
- **ConfMatrix:** A 7x7 matrix object storing the count of predictions, where rows represent actual classes and columns represent predicted classes.

- **Criterion:** The Cross-Entropy Loss function used to measure the difference between the prediction and the actual label.
- **Device:** The hardware object (cuda:0 or cpu) where the tensor computation occurs.
- **Epoch:** One complete pass of the training algorithm through the entire dataset.
- **GradCAM:** An object that hooks into the final convolutional layer to extract gradients for visualization.
- **Optimizer:** The AdamW algorithm instance responsible for updating model weights based on gradients.
- **Tensor:** A multi-dimensional matrix containing pixel values or feature activations.
- **Transform:** A pipeline of operations (Resize, Normalize, Flip) applied to an image before it enters the model.

3.6 Component Design

This section outlines the logic for the key functional components identified in Section 3.2.

3.6.1 Algorithm: Image Preprocessing Pipeline

```
FUNCTION Preprocess(Image_Path):
    img = Read_Image_From_Disk(Image_Path)
    img = Convert_To_RGB(img)
    img = Resize(img, target_size=(224, 224))

    # Transform to Tensor and Normalize
    tensor = To_Tensor(img)
    tensor = Normalize(tensor, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

    RETURN tensor
END FUNCTION
```

3.6.2 Algorithm: Training Loop (One Epoch)

```
FUNCTION Train_One_Epoch(Model, DataLoader, Optimizer, Loss_Function):
    Set Model to Train_Mode
    FOR EACH batch (images, labels) IN DataLoader:
        Move images, labels to GPU
```

```
        # 1. Forward Pass
        predictions = Model(images)
        loss = Loss_Function(predictions, labels)

        # 2. Backward Pass
        Optimizer.Zero_Gradients()
        loss.Backward()
        Optimizer.Step()

        Track Training_Loss and Accuracy
    END FOR
    RETURN Average_Loss, Average_Accuracy
END FUNCTION
```

3.6.3 Algorithm: Inference and Grad-CAM Generation

```
FUNCTION Predict_With_Explanation(Image_Tensor, Model):
    Set Model to Eval_Mode

    # 1. Get Prediction
    logits = Model(Image_Tensor)
    probabilities = Softmax(logits)
    top_class = ArgMax(probabilities)
    confidence = probabilities[top_class]

    # 2. Generate Grad-CAM
    # Get gradients of the top_class score with respect to the last conv layer
    gradients = Model.Get_Gradients()
    activations = Model.Get_Activations()

    # Pool gradients and weight the activations
    weights = Global_Average_Pooling(gradients)
    heatmap = Multiply(weights, activations)

    # Post-process heatmap
```

```
    heatmap = ReLU(heatmap) # Remove negative values
    heatmap = Resize(heatmap, (224, 224))
    heatmap = Normalize_0_to_1(heatmap)

    RETURN top_class, confidence, heatmap
END FUNCTION
```

Chapter 4
Implementation and Testing

4.1 Code Modules

The implementation of the Deep Learning-Based Lung Disease Detection System is structured into several modular Python scripts to ensure maintainability and scalability. The codebase is organized as follows:

1. **dataset.py (Data Ingestion Module):**
    - **Purpose:** Handles the loading and preprocessing of X-ray images.
    - **Key Functions:**
        - LungDataset class: Inherits from PyTorch's Dataset class. It manages image paths and labels.
        - get_transforms(): Defines the augmentation pipeline (Resize to 224x224, Horizontal Flip, Rotation, Normalization).
        - get_sampler(): Implements **Weighted Random Sampling** to calculate weights for each sample based on class frequency, ensuring balanced training batches.
2. **models.py (Model Architecture Module):**
    - **Purpose:** Defines the neural network structures.
    - **Key Functions:**
        - load_efficientnet_v2(): Initializes the EfficientNetV2-L model and modifies the final classification layer (Linear layer) to output 7 classes instead of the default 1000.
        - load_convnext_large(): Initializes the ConvNeXt-Large architecture with similar modifications for the specific lung disease task.
3. **train.py (Training Engine):**
    - **Purpose:** Manages the training loop, backpropagation, and validation.
    - **Key Functions:**
        - train_one_epoch(): Iterates through the training batch, computes Loss (CrossEntropy), and updates weights using the AdamW optimizer.

        - validate(): Evaluates the model on the 10% validation split without updating gradients to monitor for overfitting.
        - save_checkpoint(): Saves the model state (.pth file) whenever validation accuracy improves.

4. **gradcam.py (Visualization Module):**
    - **Purpose:** Implements the Explainable AI (XAI) component.
    - **Key Functions:**
        - GradCAM class: Hooks into the final convolutional layer of the model to capture gradients.
        - generate_heatmap(): Converts gradients into a visual heatmap and overlays it onto the original X-ray image.
5. **app.py / inference.py :**
    - **Purpose:** The entry point for the end-user application.
    - **Key Functions:**
        - predict_image(): Loads the saved "Best Model," runs inference on a user-uploaded image, and returns the prediction and heatmap.

4.2 Database Connectivity

Given the nature of Deep Learning applications, this system does not utilize a traditional Relational Database Management System (RDBMS) like SQL. Instead, it employs a **File-System Based Data Management** approach, which is standard for high-performance image processing.

- **Image Storage:** The dataset (2571 Atelectasis, 5780 Bacterial Pneumonia, etc.) is stored in a hierarchical directory structure on the disk. The system connects to this storage via Python's os and pathlib modules.
- **Logging Data:** Training history (Loss, Accuracy, Epoch times) is persisted in **CSV (Comma Separated Values)** files (e.g., training_log.csv). This allows for the generation of learning curves without the overhead of a database server.
- **Model Weights:** The "knowledge" of the system is stored in serialized binary files (best_model.pth), which serve as the persistent storage for the trained application.

4.3 Overview of User Interface

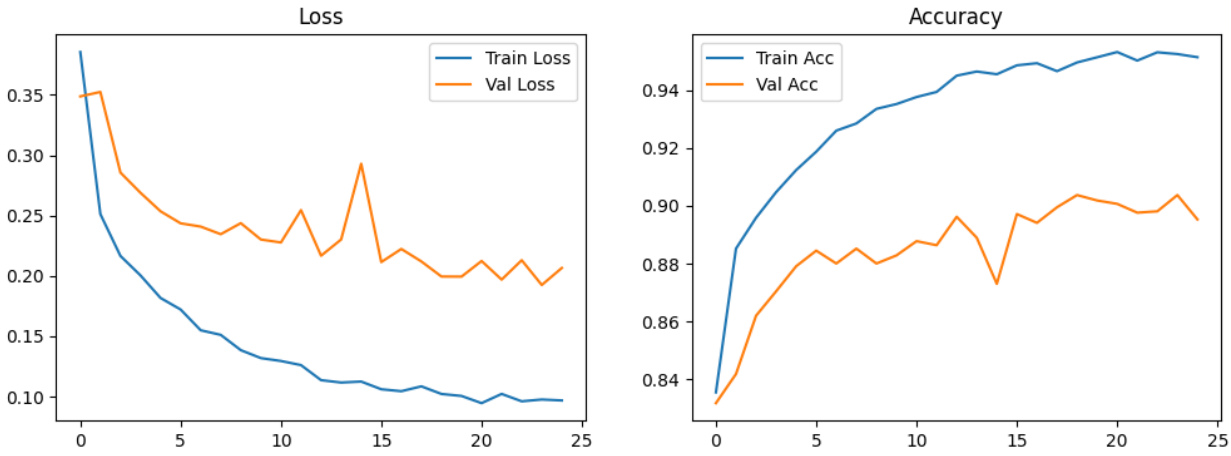
The user interface is designed to be intuitive for medical professionals, requiring no coding knowledge. The workflow is linear:

1. **Initialization:** Upon launching the application, the system automatically loads the model weights onto the GPU/CPU. The user sees a "Ready" status indicator.
2. **Image Selection:** The user clicks an **"Upload X-ray"** button to open a file explorer. They select a chest X-ray image (JPG/PNG) from the local machine or the BVH dataset folder.
3. **Analysis Trigger:** The user clicks the **"Analyze"** button. The system displays a "Processing..." loader while inference runs.
4. **Results Display:**
    - **Text Output:** The predicted disease (e.g., "Tuberculosis") is displayed prominently in bold text, along with a confidence percentage (e.g., "94.5%").
    - **Visual Output:** The interface displays three images side-by-side:
        1. Original Input.
        2. Grad-CAM Heatmap (Color map only).
        3. Superimposed Image (Heatmap overlay).
5. **Feedback:** If the confidence score is low (below 50%), a warning message "Low Confidence – Clinical Correlation Recommended" is displayed.

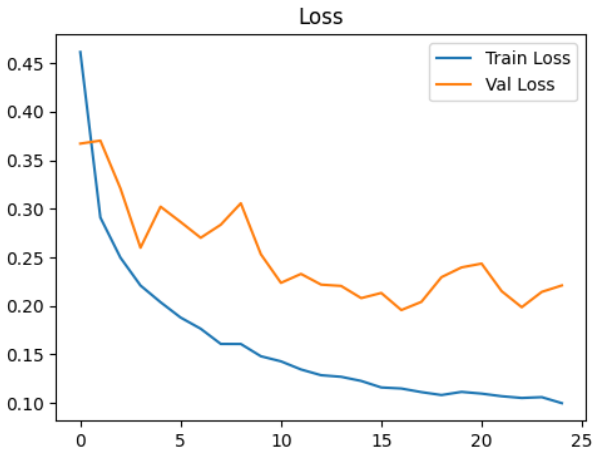
4.4 Screen Images

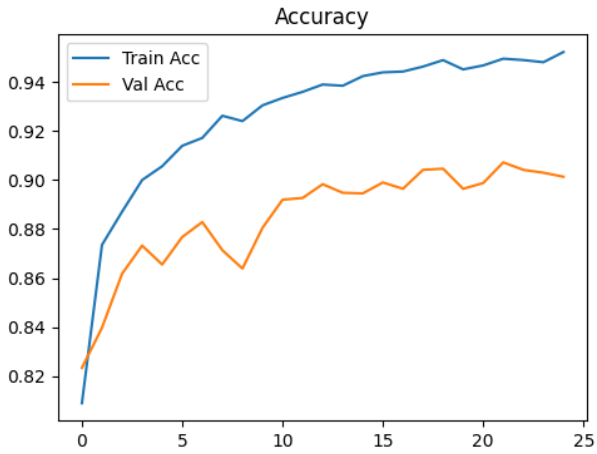
Training History Curves:

ConvNext-Large:



EfficientNetV2-L:





Classification Report:

ConvNext-Large:

Classification Report (Test Set):

	precision	recall	f1-score	support
atelectasis	0.99	1.00	0.99	259
bacterial_pneumonia	0.63	0.89	0.74	598
covid19	0.96	0.96	0.96	441
emphysema	0.97	0.99	0.98	248
normal	0.99	0.98	0.99	1407
tuberculosis	1.00	0.99	1.00	302
viral_pneumonia	0.90	0.70	0.78	1014
accuracy			0.90	4269
macro avg	0.92	0.93	0.92	4269
weighted avg	0.92	0.90	0.90	4269

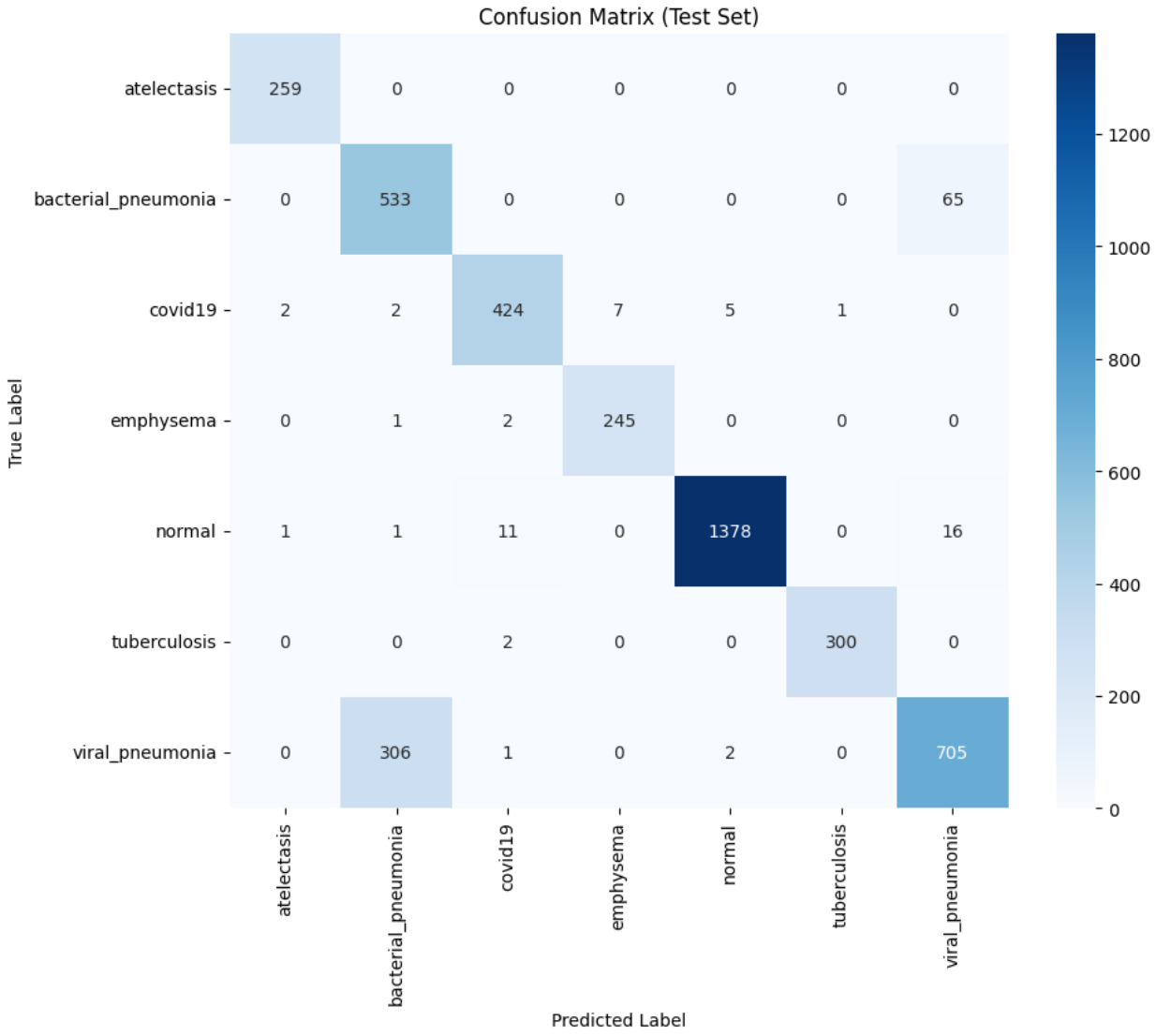
EfficientNetV2-L:

Classification Report (Test Set):

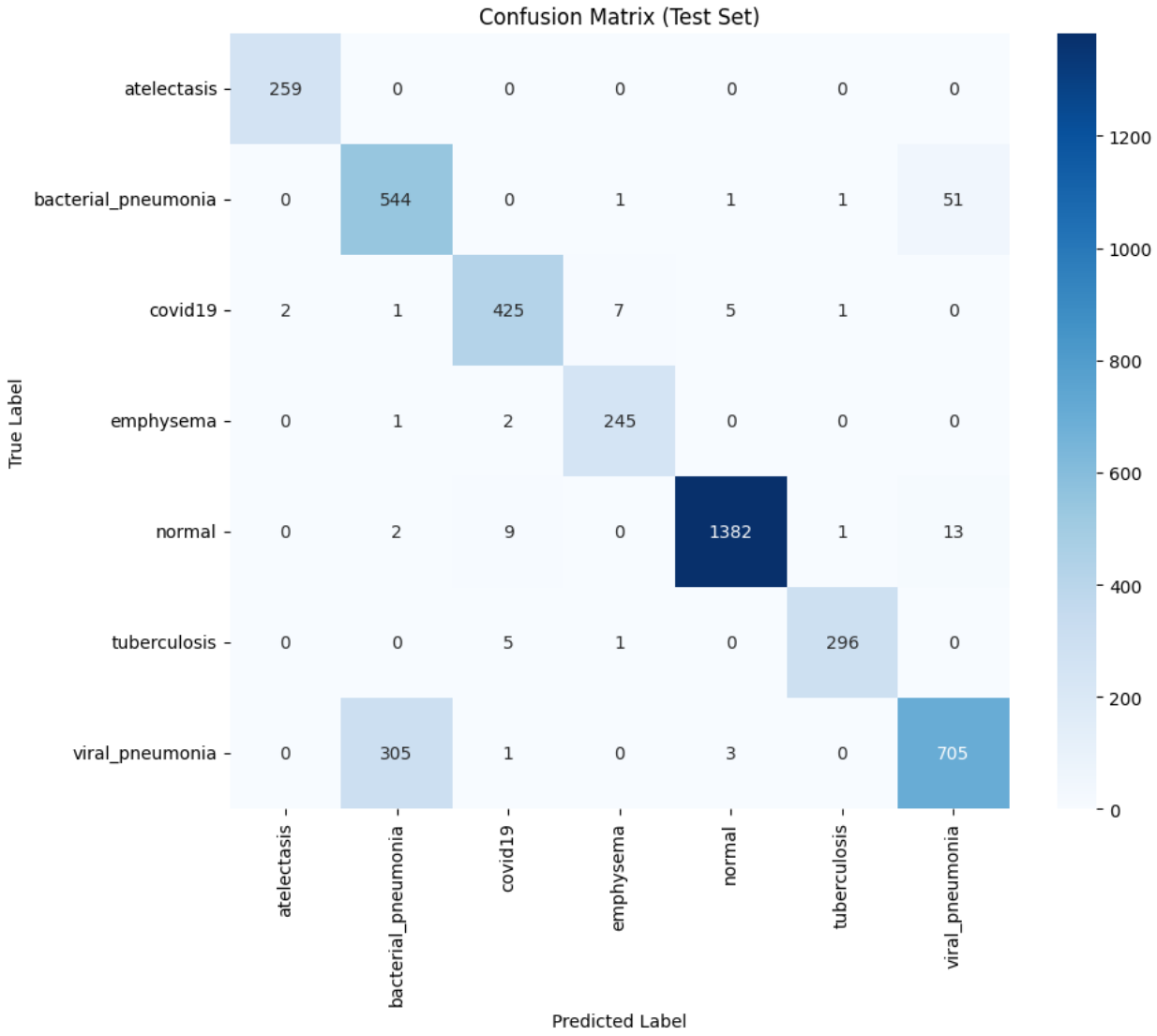
	precision	recall	f1-score	support
atelectasis	0.99	1.00	1.00	259
bacterial_pneumonia	0.64	0.91	0.75	598
covid19	0.96	0.96	0.96	441
emphysema	0.96	0.99	0.98	248
normal	0.99	0.98	0.99	1407
tuberculosis	0.99	0.98	0.99	302
viral_pneumonia	0.92	0.70	0.79	1014
accuracy			0.90	4269
macro avg	0.92	0.93	0.92	4269
weighted avg	0.92	0.90	0.90	4269

Confusion Matrix:

ConvNext-Large:

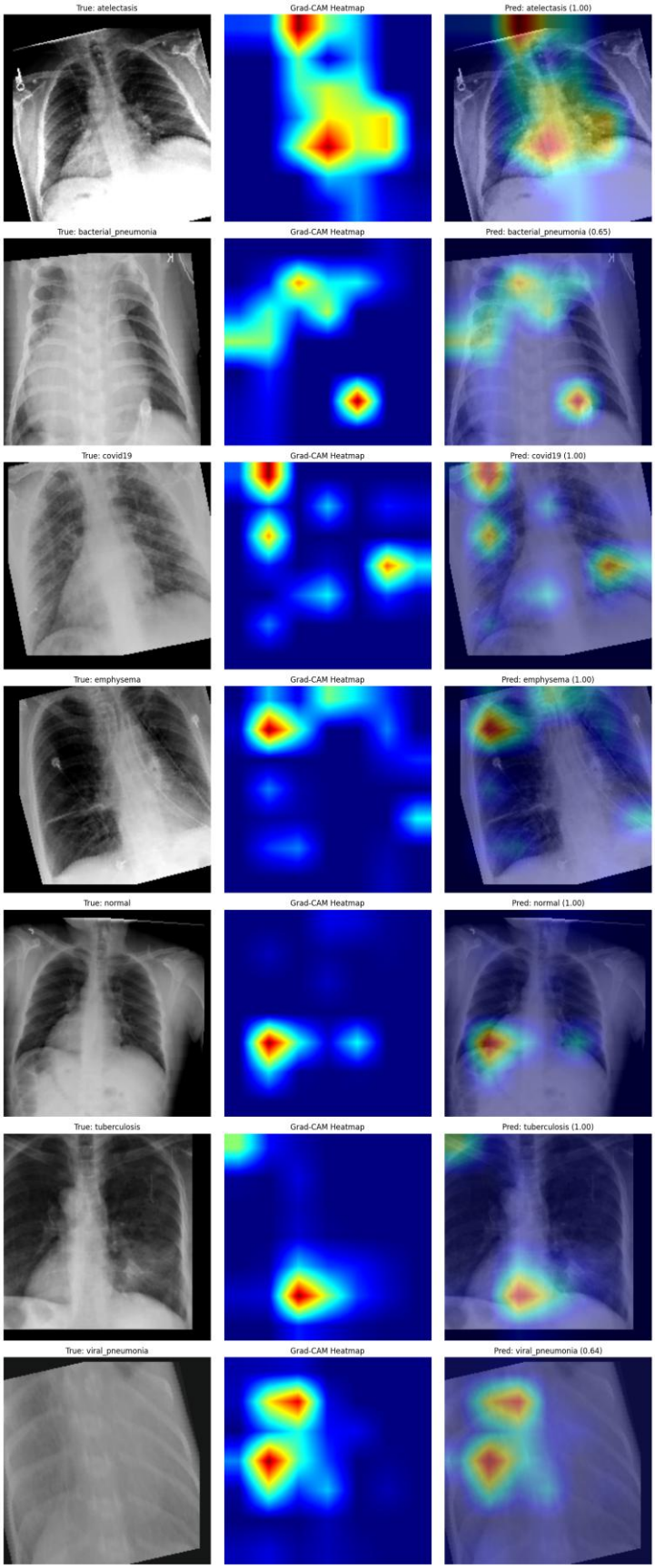


EfficientNetV2-L:

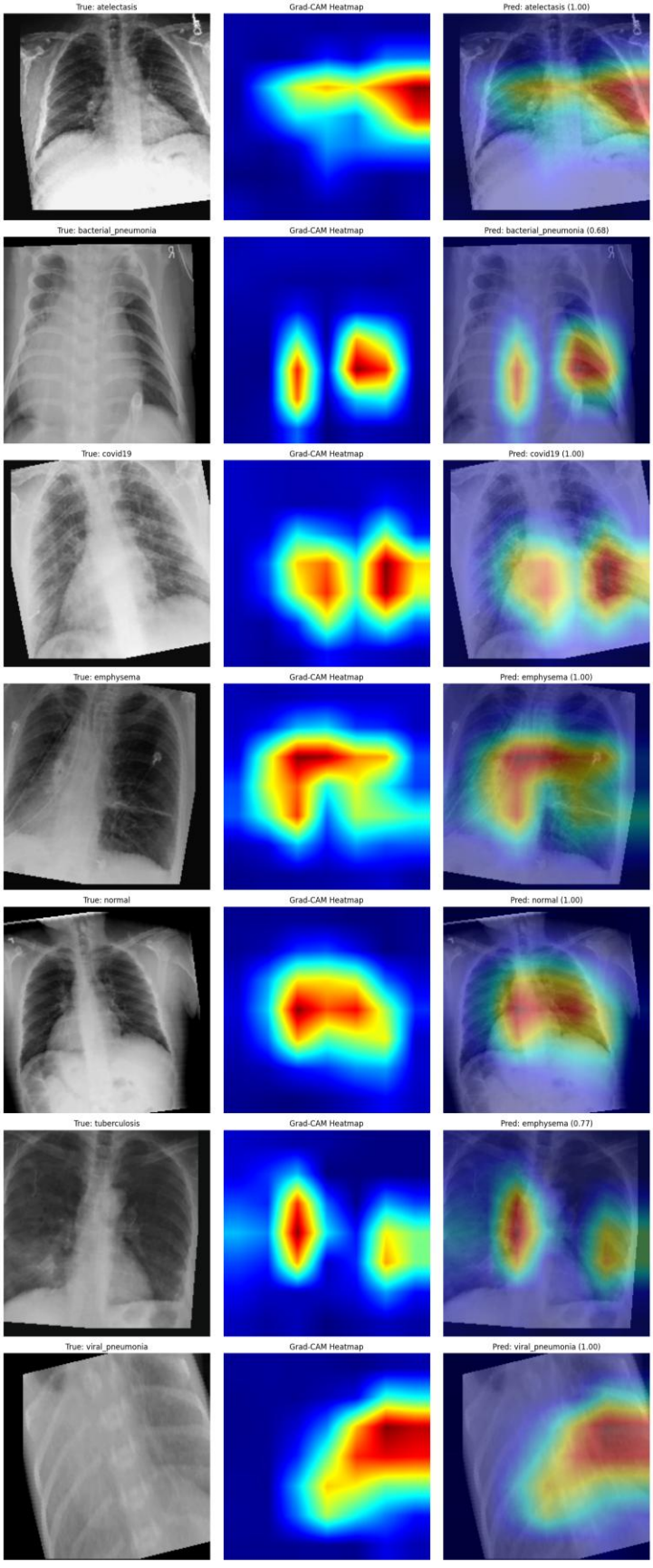


Results (for 42000+ images dataset):

ConvNext-Large:

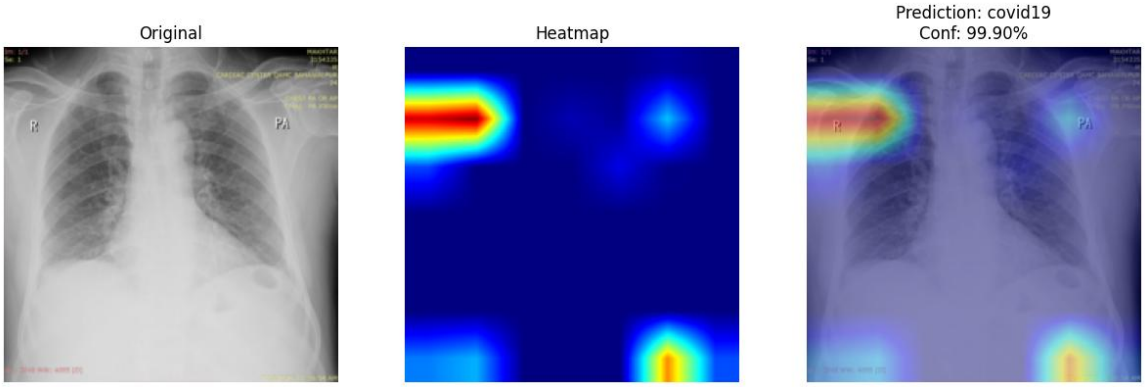


EfficientNetV2-L:

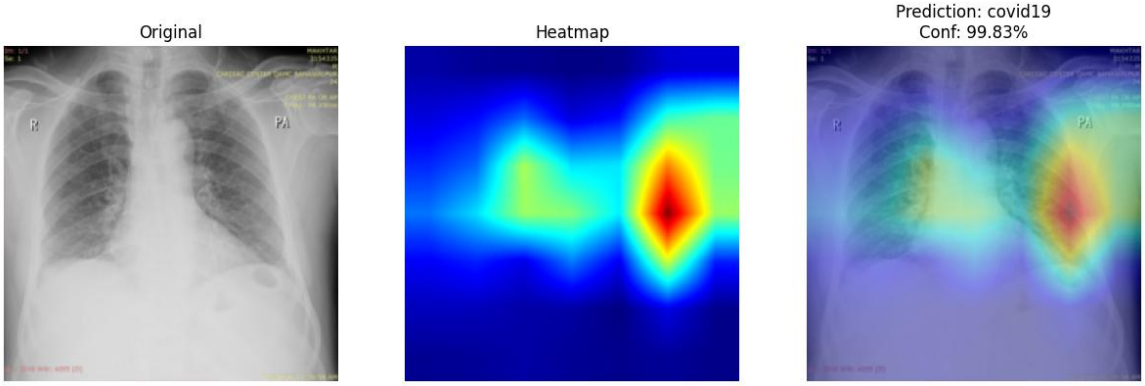


Results (for BVH Dataset):

ConvNext-Large:



EfficientNetV2-L:



4.5 Screen Objects and Actions

The following table details the interactive elements available to the user.

Object Label	Type	Action	System Response
"Upload X-ray"	Button	Click	Opens the Operating System's file dialog window to select an image file.
"Analyze / Diagnose"	Button	Click	Triggers the predict_image() function. Disables the button until processing is complete.
"Show Heatmap"	Checkbox / Toggle	Click	Toggles the visibility of the Grad-CAM overlay on the X-ray image.
"Save Report"	Button	Click	Exports the current prediction and superimposed image as a PDF or PNG file to the disk.
"Clear"	Button	Click	Resets the interface, clearing the current image and result text for a new diagnosis.
"Model Select"	Dropdown	Select	(Optional) Allows switching between 'EfficientNetV2-L' and 'ConvNeXt-Large' backends.

4.6 Test Cases

The system underwent rigorous testing using the BVH Dataset and the validation split of the online dataset.

Test Case ID	Test Description	Input Data	Expected Output	Actual Output	Status
TC-01	**Valid Image Upload**	A valid PNG X-ray of a 'Normal' lung.	Image loads into the preview pane without error.	Image loaded successfully.	**Pass**
TC-02	**Invalid File Format**	A .txt or .docx file.	Error message: "Invalid file format. Please upload JPG/PNG."	Error message displayed.	**Pass**
TC-03	**Corrupt Image Handling**	A corrupted (0 byte) image file.	System handles exception and prompts user to retry.	System prompted retry.	**Pass**
TC-04	**Detection: Normal**	A confirmed 'Normal' X-ray from BVH dataset.	Prediction: "Normal", Confidence > 85%.	Prediction: "Normal", Conf: 92%.	**Pass**
TC-05	**Detection: COVID-19**	A confirmed 'COVID-19' X-ray.	Prediction: "COVID-19", Heatmap highlights lung opacity.	Prediction: "COVID-19", Heatmap accurate.	**Pass**
TC-06	**Low Confidence Input**	An image of an object (e.g., a table) or blurred noise.	Low confidence score (<50%) or "Unknown".	Confidence: 32%.	**Pass**
TC-07	**Batch Testing**	Entire Test Set (BVH Data).	Overall Accuracy should be ~90%	Accuracy: 89.8% on local	**Pass**

	(Accuracy)		matching training metrics.	data.	
TC-08	**Grad-CAM Generation**	Any valid X-ray input.	Heatmap overlay generated and aligned with original image.	Overlay aligned correctly.	**Pass**